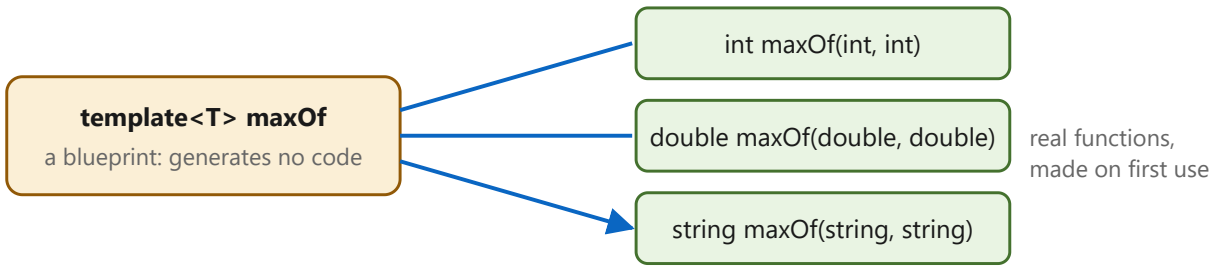
C++ Templates

Write code once, let the compiler generate it per type. From a first function template to forwarding references, variadics and type traits: the topics the job description lists.

Why templates, and function templates



Compile-time polymorphism: no runtime cost, more code per type.

Without templates: `maxInt`, `maxDouble`, `maxString`... A **template** takes the type as a parameter:

```
template <typename T>
T maxOf(T a, T b) {
    return a < b ? b : a;
}

maxOf(3, 7);           // compiler deduces T = int
maxOf(2.5, 1.5);       // T = double
maxOf<double>(3, 2.5);// explicit: T = double, the int is converted
```

The template is not compiled itself. Each new type **instantiates** a real function: no runtime cost, more binary size, and errors appear only when a bad type is used.

KEY POINT Templates: **compile-time** (static) polymorphism, fastest, inlinable, more code, types known at compile time. Virtual: **runtime**, one code path, mixed objects, a vtable indirection.

Template argument deduction

When you call a function template, the compiler works out `T` from the arguments. The rule depends on the **shape of the parameter**, and it is the same rule `auto` uses.

PARAMETER	CALL	DEDUCED `T`	RULE
`T x` (by value)	`f(ci)` with `const int ci`	`int`	a copy: top-level `const` and references are **dropped**
`T x` (by value)	`f(arr)` with `int arr[3]`	`int*`	arrays and functions **decay** to pointers
`T& x`	`f(ci)`	`const int`	the reference keeps the argument's `const`
`T& x`	`f(arr)`	`int[3]`	an array keeps its size (usable to get its length)
`const T& x`	`f(5)`	`int`	binds temporaries; `const` is not part of `T`
`T&& x`	`f(i)` / `f(5)`	`int&` / `int`	forwarding reference (see Forwarding references)

Compiled and checked with `static_assert`: by value, `const int` deduced `int` and the array deduced `int*`; by reference they deduced `const int&` and `int(&)[3]`.

- Deduction looks only at the **arguments**, never at the return type, and **no conversions** happen: `maxOf(3, 2.5)` fails because `T` would be both `int` and `double`.
- Default arguments and braced lists do not take part in deduction: `f({1, 2})` cannot deduce `T`. Name `T` explicitly: `f<std::vector<int>>({1, 2})`.
- Name the type yourself when needed: `maxOf<double>(3, 2.5)`.

Class template argument deduction (C++17)

```cpp
std::pair p{1, 2.5};                    // std::pair<int, double>
std::vector v{1, 2, 3};                 // std::vector<int>
std::array a{1, 2, 3};                  // std::array<int, 3>

template <typename T> struct Box { T value; };
// a user-defined deduction guide (needed for aggregates before C++20)
template <typename T> Box(T) -> Box<T>;
Box b{std::string("x")};                // Box<std::string>
```

The compiler deduces the class's arguments from the constructor call. A **deduction guide** tells it how when the constructors alone are not enough. All of the above was compiled and checked.

> **WHEN DEDUCTION FAILS** The error says "couldn't deduce template parameter". Look for conflicting arguments, a braced list, or a parameter that does not mention `T` at all (e.g. only the return type does).

Class templates

Same idea for classes: `vector<T>`, `map<K,V>`, `unique_ptr<T>`.

```cpp
// a type parameter, a NON-TYPE parameter, with a default
template <typename T, std::size_t Capacity = 8>
class Stack {
    std::array<T, Capacity> items_;
    std::size_t size_ = 0;
public:
    bool push(const T& v) {
        if (size_ == Capacity) return false;
        items_[size_++] = v;
        return true;
    }
    T pop() { return items_[--size_]; }
    bool empty() const { return size_ == 0; }
};

Stack<int> a;                 // Stack<int, 8>
Stack<std::string, 100> b;    // a different, unrelated type
```

```cpp
// C++17 CTAD: class template argument deduction → std::pair<int, double>
std::pair p{1, 2.5};
std::vector v{1, 2, 3};     // std::vector<int>
```

- **Non-type parameters** are compile-time constants (how `std::array<int,5>` knows its size).
- Each instantiation is a separate, unrelated class. Members are instantiated only if called.

> **WHY TEMPLATES LIVE IN HEADERS** The compiler needs the body at instantiation. In a `.cpp`, others see only a declaration: linker errors. Use headers, or explicit instantiation (`template class Stack<int>;`).

Specialization

A **specialization** replaces the generic template for specific arguments.

```cpp
template <typename T>
struct Printer {                            // primary template
    static void print(const T& v) { std::cout << v << '\n'; }
};

template <>                                 // full specialization for bool
struct Printer<bool> {
    static void print(bool v) { std::cout << (v ? "yes" : "no") << '\n'; }
};

template <typename T>
// PARTIAL specialization: any pointer type
struct Printer<T*> {
    static void print(T* p) { std::cout << "ptr to " << *p << '\n'; }
};
```

KIND	MEANING	ALLOWED FOR
Full specialization	all parameters fixed: `template<> ... Printer<bool>`	class and function templates
Partial specialization	a pattern: `Printer<T*>`	**class templates only**

> **FUNCTIONS: PREFER OVERLOADS** No partial specialization for function templates. Overload instead; an exact non-template match wins. Function specializations do not take part in overload resolution.

Variadic templates and fold expressions

A **parameter pack** (`typename ... Args`) takes any number of arguments; `args ...` expands it. Before C++17: recursion with a base case.

```
template <typename T>
// base case: one argument
T sum(T v) { return v; }

template <typename T, typename ... Rest>
// peel one off, recurse
T sum(T first, Rest ... rest) { return first + sum(rest ...); }

sum(1, 2, 3);
```

The compiler instantiates a chain of real functions.

C++17 **fold expressions** replace the recursion:

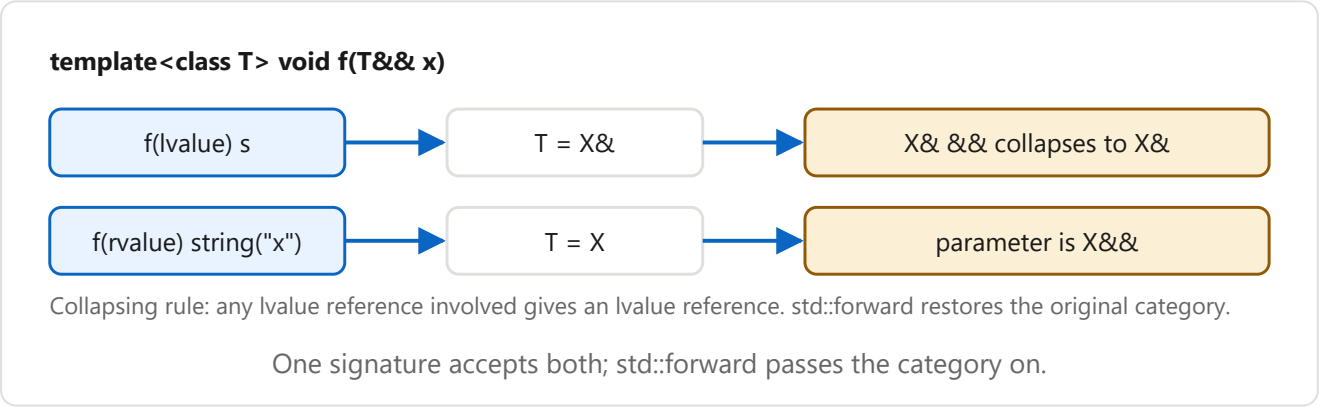
```
template <typename ... Args>
// unary right fold: a + (b + (c ...))
auto sum(Args ... args) { return (args + ...); }

template <typename ... Args>
// comma fold: do it for each arg
void print(Args&& ... args) { ((std::cout << args << ' '), ...); }

// number of elements in the pack
sizeof ...(Args);
```

Behind `make_unique`, `make_shared`, `emplace_back`, `tuple`, `thread`, `function`.

Forwarding references and perfect forwarding



`T&&` **in a function template** is a **forwarding reference**, not an rvalue reference: it binds to anything and remembers lvalue vs rvalue.

Is it really a forwarding reference?

Only when `T` is deduced for *that* function and the form is exactly `T&&` or `auto&&`:

DECLARATION	FORWARDING REFERENCE?
`template<class T> void f(T&& x)`	yes: `T` is deduced for `f`
`auto&& x = expr;`	yes
`template<class T> void f(std::vector<T>&& v)`	no: not exactly `T&&`, so it binds rvalues only
`template<class T> void f(const T&& x)`	no: `const` disqualifies it
`template<class T> struct S { void f(T&& x); };`	no: `T` is fixed by the class, so nothing is deduced for `f`
`void f(Widget&& w)`	no: a plain rvalue reference

Reference collapsing

Reference to reference collapses: **any lvalue reference makes it an lvalue reference; only `&& + &&` stays rvalue.**

WRITTEN	COLLAPSES TO
`T& &`, `T& &&`, `T&& &`	`T&`
`T&& &&`	`T&&`

Perfect forwarding

A wrapper's parameter has a *name*, so it is an lvalue and the callee would always copy. `std::forward` restores the category:

```
template <typename T, typename... Args>
// forwarding references
std::unique_ptr<T> makeUnique(Args&&... args) {
    // lvalues stay lvalues, rvalues stay rvalues
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}

std::string s = "abc";
// s is an lvalue → Widget copies it
auto a = makeUnique<Widget>(s);
// temporary rvalue → Widget MOVES it
auto b = makeUnique<Widget>(std::string("x"));
```

> **STD::MOVE VS STD::FORWARD** `move` *always* casts to rvalue. `forward<T>` casts to rvalue *only if* the argument was one. Use `forward` only on a forwarding reference, once.

Alias templates, variable templates and more

TOOL	EXAMPLE	USE
Alias template	`template <typename T> using Table = std::map<std::string, std::vector<T>>;`	a shorter name that still takes a parameter; unlike `typedef` it can be templated
Variable template (C++14)	`template <typename T> constexpr T zero = T{};`	a constant per type: `zero<int>`, `zero<double>`. The standard's `std::is_integral_v<T>` is one
Template template parameter	`template <template <typename ...> class C, typename T>`	pass a *template* (`std::vector`) rather than a type: `makeTwo<std::vector>(3, 4)`
Non-type parameter	`template <typename T, std::size_t N>`	a value as parameter; `auto` accepts any type of value in C++17: `template <auto V>`
Abbreviated template (C++20)	`void f(auto x)`	each `auto` parameter is an implicit template parameter

```
template <typename T> using Table = std::map<std::string, std::vector<T>>;
// std::map<std::string, std::vector<int>>
Table<int> t;

// zero<int> is 0, zero<double> is 0.0
template <typename T> constexpr T zero = T{};

template <template <typename ...> class Container, typename T>
Container<T> makeTwo(T a, T b) { Container<T> c; c.push_back(a);
c.push_back(b); return c; }
// a vector<int> with two elements
auto v = makeTwo<std::vector>(3, 4);
```

Unpacking with `std::index_sequence`

To call a function with the elements of a `std::tuple` you need the indices `0, 1, 2 ...` as a pack. `std::make_index_sequence<N>` generates it:

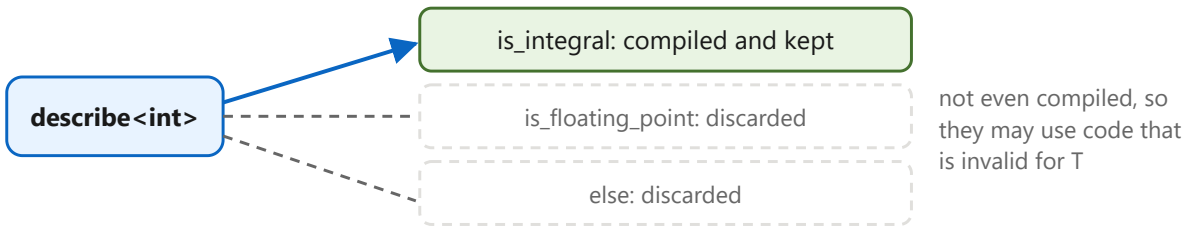
```
template <typename F, typename Tuple, std::size_t ... I>
// expands to get<0>, get<1>, ...
auto applyImpl(F f, const Tuple& t, std::index_sequence<I ... >) { return
f(std::get<I>(t) ... ); }

template <typename F, typename Tuple>
auto applyTuple(F f, const Tuple& t) { return applyImpl(f, t,
std::make_index_sequence<std::tuple_size_v<Tuple>>{}); }

// 6 (compiled)
applyTuple([](int x, int y, int z) { return x + y + z; }, std::tuple{1, 2,
3});
```

C++17 ships this as `std::apply`. Know the idea: a pack of integers lets you expand an expression once per element.

Type traits and `if constexpr`



A normal if would fail to compile as soon as one branch is invalid for T.

A template often needs to ask about its type. `<type_traits>` answers at compile time (`std::is_integral_v<T>`), and `if constexpr` (C++17) keeps only the matching branch:

```
template <typename T>
std::string describe(const T& v) {
    if constexpr (std::is_integral_v<T>)            return "integer "   +
std::to_string(v);
    else if constexpr (std::is_floating_point_v<T>) return "float "     +
std::to_string(v);
    else                                            return "something else";
}
```

Traits, SFINAE and `enable_if`, concepts, `constexpr`, `consteval` and template metaprogramming have their own lesson: Type Traits & Compile-time C++.

Pitfalls and good practice

PITFALL	EXPLANATION / FIX
Huge unreadable error messages	The error is reported deep inside an instantiation. Read the *first* error and the line "required from here". Concepts or `static_assert` on traits give early, clear errors.
Template body in a `.cpp` file	Linker errors. Put the definition in the header.
`typename` for dependent types	Inside a template, `T::iterator` could be a type or a value. Write `typename T::iterator it;`.
Members of a dependent base class not found	Write `this->member` or `Base<T>::member`.
Code bloat	Every distinct `T` generates more machine code. Move type-independent logic into a non-template helper.
Deduction conflicts: `maxOf(3, 2.5)`	Two different deduced `T`s fail. Cast, specify `maxOf<double>`, or use two parameters.
`maxOf("a", "b")` with `const char*`	Compares pointer addresses, not text. Use `std::string` or `std::string_view`.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Why templates, and function templates	C++ Primer	16.1 Defining a template	681 to 706
Template argument deduction	C++ Primer	16.2 Template argument deduction	707 to 722
	C++ Primer	16.3 Overloading and templates	723 to 727
	Effective Modern C++	Item 1, Understand template type deduction	27 to 36
Class templates	C++ Primer	16.1 Defining a template	681 to 706
	Programming: Principles and Practice	18.1 to 18.3 Templates, generalizing Vector, range checking and exceptions	527 to 541
Specialization	C++ Primer	16.5 Template specializations	735 to 743
Variadic templates and fold expressions	C++ Primer	16.4 Variadic templates	728 to 734
Forwarding references and perfect forwarding	C++ Primer	16.2 Template argument deduction	707 to 722
	Effective Modern C++	Item 23, Understand std::move and std::forward	176 to 182
	Effective Modern C++	Item 24, Distinguish universal references from rvalue references	182 to 186
	Effective Modern C++	Items 26 to 27, and 30 Overloading on universal references; perfect forwarding failure cases	195 to 215 and 225 to 232
	C++ Move Semantics	9.1 to 9.3 Perfect forwarding; universal versus rvalue references	159 to 166
	C++ Move Semantics	9.4 to 9.5 Overload resolution with universal references; perfect forwarding in lambdas	167 to 170
	C++ Move Semantics	Chapter 11 Perfect passing with auto&&	191 to 202
Type traits and if constexpr	Effective Modern C++	Items 26 to 27, and 30 Overloading on universal references; perfect forwarding failure cases	195 to 215 and 225 to 232

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
	C++ Move Semantics	Chapter 12 Perfect returning with decltype(auto)	203 to 210